AI PRINCIPLES & TECHNIQUES

PROGRAMMING ASSIGNMENT 1

Levente Bódi	s1122062
Konstantinos Konstantinou	s1118596

Introduction

Search algorithms play a significant role in artificial intelligence as they can be applied to a wide variety of optimization and decision-making problems. One of their many applications is in finding optimal strategies in games via efficiently navigating through large datasets represented as game trees. In this report, we focus on the practical implementation of one such algorithm, namely the ***MinMax algorithm***, and its optimized variant, ***alpha-beta pruning***, for the game “N in a row”.

The MinMax algorithm is a decision-making algorithm used in two-player games, where one player aims to maximize their score while the other tries to minimize it. The algorithm explores all possible moves by constructing a game tree, assuming that at each step both players will play optimally, choosing the best possible outcome for themselves.

Alpha-beta pruning is an improvement on the MinMax algorithm that reduces the number of nodes evaluated in a tree. It achieves this by eliminating branches that cannot influence the final decision, thereby improving efficiency.

To familiarize ourselves with the concepts of efficiency and time complexity, our task was to implement these algorithms for the game and experiment with different configurations, including various board sizes, search depths, and "N in a Row" lengths. We then compared the runtime performance of the MinMax algorithm with and without alpha-beta pruning.

Method

MinMax Algorithm

As aforementioned, the Minimax algorithm is designed to explore all possible game states, assuming that both players will play optimally. In order to implement this, a tree structure is required to represent all game states. In this tree structure, each node corresponds to a particular game state, and the edges between nodes represent the possible moves that can be made from one state to another. The tree is built **top down**, beginning from the current game state - referred to as the **root node** - and expands to a specified depth, or until all leaf

nodes are terminal game states. **Leaf nodes** represent the terminal states of the game or states beyond which further exploration is unnecessary due to depth limits.

Each node in this tree structure is evaluated based on a **heuristic function**, which assigns a score to the node depending on the state of the game; whether a player is winning, losing, or in a neutral state. Unlike the top-down construction of the game tree, the evaluation is performed **bottom-up**, meaning that the algorithm starts by evaluating the leaf nodes first and then proceeds upwards toward the root node. The MinMax algorithm operates under the following two assumptions:

- The **maximizing player** will always try to maximize the score, i.e., select the move that yields the highest possible value.
- The **minimizing player** will try to minimize the score, i.e., select the move that yields the lowest possible value.

At each level of the tree, as the players' turns alternate, the algorithm alternates between the maximizing and minimizing player. This ensures that the strategy accounts for the best possible outcome for both players, assuming that both are playing optimally, as explained earlier. When the algorithm reaches the root node during evaluation, it selects the optimal move for the player whose turn it is based on the evaluated outcomes. Specifically:

- If it is the maximizing player's turn, the algorithm selects the move corresponding to the **highest** evaluated score.
- If it is the minimizing player's turn, the algorithm selects the move corresponding to the **lowest** evaluated score.

```
function Minimax(node, depth, maximizingPlayer):
    if depth == 0 or node is a terminal node:
        return heuristic value of node

    if maximizingPlayer:
        maxEval = -infinity
        for each child of node:
            eval = Minimax(child, depth - 1, False)
            maxEval = max(maxEval, eval)
        return maxEval
    else:
        minEval = infinity
        for each child of node:
            eval = Minimax(child, depth - 1, True)
            minEval = min(minEval, eval)
        return minEval
```

MinMax Pseudocode

Alpha-Beta Pruning

Alpha-beta pruning enhances the efficiency of the MinMax algorithm by reducing the number of nodes that need to be evaluated in the game tree. Unlike the standard MinMax, which explores all possible game states, Alpha-beta pruning **prunes** - eliminates or cuts off - branches that cannot affect the final outcome, significantly improving performance, particularly in deeper trees. The algorithm achieves this by maintaining two values during its traversal of the tree: **alpha** and **beta**. These values represent the best scores achievable for the **maximizing** and **minimizing** players, respectively:

- **Alpha**: The highest score that the maximizing player is guaranteed to achieve.
- **Beta**: The lowest score that the minimizing player is guaranteed to achieve.

As the algorithm traverses the game tree, it updates alpha and beta at each node to reflect the best possible outcome seen so far for both players. If, during traversal, the maximizing player finds a move with a value greater than or equal to beta, the remaining branches are pruned. Similarly, if the minimizing player finds a value less than or equal to alpha, the remaining branches are pruned. This pruning avoids unnecessary evaluations, reducing the overall complexity.

Like MinMax, Alpha-beta pruning follows a **bottom-up** evaluation. It starts by evaluating the leaf nodes with a heuristic function and then works upwards toward the root, updating alpha and beta values along the way. By pruning suboptimal branches, the algorithm can focus on the most promising moves, further optimizing the decision-making process.

```
function AlphaBeta(node, depth, alpha, beta, maximizingPlayer):
  if depth == 0 or node is a terminal node:
    return heuristic value of node

  if maximizingPlayer:
    maxEval = -infinity
    for each child of node:
      eval = AlphaBeta(child, depth - 1, alpha, beta, False)
      maxEval = max(maxEval, eval)
      alpha = max(alpha, eval)
      if beta <= alpha:
        break  # beta cut-off
    return maxEval
  else:
    minEval = infinity
    for each child of node:
      eval = AlphaBeta(child, depth - 1, alpha, beta, True)
      minEval = min(minEval, eval)
      beta = min(beta, eval)
      if beta <= alpha:
        break  # alpha cut-off
    return minEval
```

Alpha-Beta Pruning Pseudocode

Time complexity

The MinMax algorithm explores the entire game tree up to a certain depth. If b is the branching factor, representing the average number of moves available at each node, and d is the depth of the tree, then the time complexity of the algorithm is O(b^d). This exponential growth results from the fact that at each level of depth, the number of nodes increases by a factor of b.

A more efficient version of the MinMax algorithm is Alpha-Beta pruning. In the worst-case scenario, the time complexity of Alpha-Beta pruning is the same as MinMax, O(b^d). However, in the best-case scenario, Alpha-Beta pruning achieves O($b^{d/2}$) by pruning branches that cannot affect the final decision. This considerable improvement effectively reduces the number of nodes evaluated by half, compared to the MinMax algorithm. If the worst moves are evaluated first, the pruning may be less effective, resulting in less significant improvements.

Results

In order to validate the efficiency and complexity of the MinMax algorithm and its optimized counterpart, Alpha-Beta pruning, a series of experiments were conducted with varying parameters, including the number of symbols in a row to win (denoted as N), board size, and search depth. The runtime, node expansions, and branching factors were measured for each algorithm across different configurations.

Explanation of Practical Measures

Before delving into the detailed results, it is important to clarify the metrics used to compare the two algorithms:

- **Time Measure**: While actual runtime (measured in seconds) provides a practical sense of how long each algorithm takes to execute, it is not always the most reliable standalone measure of efficiency. Runtime can be influenced by external factors like CPU load or system performance. Therefore, additional metrics were considered to provide a clearer picture of the inherent computational complexity of the algorithms.
- **Node Expansions**: This metric represents the number of nodes expanded during the search process. The number of node expansions provides a direct measurement of the computational effort required by each algorithm
- **Branching Factor**: The branching factor refers to the average number of children generated per node at each level of the search tree. This metric provides insight into the complexity of the game tree structure

The following table summarizes the experimental results based on these three measures:

N	Board Size	Search Depth	Algorithm	Time (s)	Nodes Expanded	Branching Factor
2	7x6	6	MinMax	2.562	25817	4302.833
2	7x6	6	AlphaBeta	1.868	23	3.833
4	7x6	6	MinMax	5.073	137257	22876.167
4	7x6	6	AlphaBeta	1.981	1306	217.667
6	7x6	6	MinMax	4.62	137257	22876.167
6	7x6	6	AlphaBeta	2.17	4945	824.167
4	5x6	6	MinMax	1.194	19531	3255.167
4	5x6	6	AlphaBeta	0.222	546	91.0
4	9x6	6	MinMax	19.219	597871	99645.167
4	9x6	6	AlphaBeta	8.346	2602	433.667
4	7x4	6	MinMax	4.632	136991	22831.833
4	7x4	6	AlphaBeta	2.28	1218	203.0
4	7x8	6	MinMax	6.227	137257	22876.167
4	7x8	6	AlphaBeta	1.948	1307	217.833
4	7x6	2	MinMax	2.877	57	28.5
4	7x6	2	AlphaBeta	1.976	21	10.5
4	7x6	4	MinMax	3.145	2801	700.25
4	7x6	4	AlphaBeta	1.849	179	44.75
4	9x8	8	MinMax	90.151	597871	74733.875
4	9x8	8	AlphaBeta	8.791	1639	204.875
4	11x10	10	MinMax	378.866	1948717	194871.7
4	11x10	10	AlphaBeta	27.259	2927	292.7

The results clearly show that **Alpha-Beta pruning consistently performs faster than MinMax**, particularly as the search depth increases. For instance, with **N = 4**, a **9x8 board**, and a **depth of 8**, MinMax takes **90.151 seconds**, while Alpha-Beta pruning takes only **8.791 seconds**. Similarly, for larger board sizes (e.g., 11x10), MinMax takes significantly longer to complete due to the exponential growth of nodes with depth.

MinMax explores every possible move, leading to a higher number of node expansions. As seen in the table, MinMax consistently expands tens of thousands or even millions of nodes for larger configurations (e.g., 1,948,717 nodes for N = 4, board size 11x10, and depth 10). In contrast, **Alpha-Beta pruning dramatically reduces the number of nodes expanded** by pruning irrelevant branches. For the same configuration (N = 4, board size 11x10, depth 10), Alpha-Beta pruning expands only **2,927 nodes**, showing a **reduction by several orders of magnitude**. This reflects the best-case scenario for Alpha-Beta pruning, where many branches are pruned early, allowing the algorithm to focus on the most promising paths.

Alpha-Beta pruning also leads to a **lower branching factor** compared to MinMax. For instance, with **N = 4**, a **9x6 board**, and a **depth of 6**, MinMax has a branching factor of **99,645.167**, while Alpha-Beta pruning has a much smaller branching factor of **433.667**. This shows how pruning can significantly reduce the number of paths explored in the game tree, particularly when it is highly effective.

Analysis of Results

- **Increasing N (number of symbols in a row to win)**: As the number of symbols required to win increases, there is a slight increase in the runtime for MinMax, while the runtime for Alpha-Beta pruning remains relatively stable. This suggests that **Alpha-Beta pruning is less sensitive to changes in N**. This seems counterintuitive at first sight, however, if we refer back to the part of Big-O complexity of both of the algorithms, so runtime is only influenced by b and d, thus, the branching factor and depth. If N is changed, the game might run longer, since the players have to connect more symbols, but Big-O remains the same. For us, it was also counterintuitive at first, thus, this was a surprising finding.
- **Board Size**: Increasing the board size significantly affects both algorithms. Larger boards result in a higher branching factor, which directly increases the number of nodes expanded. MinMax, which evaluates every possible move, suffers from this, leading to longer runtimes. **Alpha-Beta pruning mitigates this effect** by pruning irrelevant branches, resulting in lower runtimes and fewer node expansions.
- **Search Depth**: As expected, increasing the search depth leads to an exponential increase in runtime and node expansions for both algorithms. However, Alpha-Beta pruning remains more efficient, as shown by the reduced number of nodes expanded and shorter runtimes across all depths. For example, with **N = 4**, **board size 7x6**, and a **depth of 4**, MinMax expands **2,801 nodes** while Alpha-Beta pruning expands only **179 nodes**, representing a significant reduction in complexity.

Discussion

One limitation of our implementation is the reliance on a fixed heuristic for node evaluation. The quality of this heuristic significantly impacts the decisions made by both MinMax and Alpha-Beta pruning, particularly in limited-depth searches.
Another factor is the variation in Alpha-Beta pruning's effectiveness depending on move order. If stronger moves are evaluated first, pruning is highly efficient. However, if weaker moves are explored early, pruning becomes less effective, and performance approaches that of MinMax.

Conclusion

The results demonstrate that Alpha-Beta pruning is more efficient than MinMax, reducing both runtime and the number of nodes expanded, especially at deeper search depths. By pruning unnecessary branches, Alpha-Beta pruning consistently lowers the branching factor compared to MinMax, confirming its theoretical advantage. Overall, it proves to be the better choice for decision-making in larger and more complex game scenarios.